

Final Project Report

Topic 4 — Async Research Assistant

Team: Team1

Date: 19 September 2026

Topic: 4 — Async Research Assistant

Tag: v1.0-final

Repo: github.com/Nurana100/team1-ai-eng-research-assistant

Members: Nurana Aliyeva (@Nurana100), Elshan Shukurov (@elsensukurov), Ali Hasanli (@ali-hasanli), Gasham Huseynli (@gashamhuseynli)

Executive Summary

We built an async research assistant: the user asks one natural-language question at the command line, the system fans out to Wikipedia, arXiv and a web-search backend at the same time, and a single LLM call turns whatever came back into one answer in which every claim carries a citation marker pointing at a real URL. The concurrency design is validated by measurement rather than assertion — against three live sources (Wikipedia, arXiv and DuckDuckGo) the concurrent fan-out averaged 0.57s against 1.19s for the same fetches issued one after another, a 2.10× speedup over three runs (per-run range 1.78–3.29×). The theoretical ceiling for this workload is about 2.7×, so the 3.29× run is a measurement artifact, not a result. The headline robustness result is that the system keeps answering when a source dies: during benchmarking Wikipedia began returning 403 to every request, and because `asyncio.gather(..., return_exceptions=True)` sits under a per-source timeout the user still received a correctly-cited arXiv-only answer instead of a traceback — graceful degradation verified against a live failure rather than a simulated one. We later met an intermittent arXiv 406 and Gemini’s free-tier daily quota (429); the first produced a degraded answer and the second produced one clean **Error:** line. The most surprising thing we learned was how easily a benchmark misleads: a run that reports more than the arithmetic ceiling means the harness is wrong, not the code. If we started over we would build the measurement harness on day one rather than at the end, because every performance claim in this report is only as good as the harness that produced it.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing and scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 OOP design and key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency and Performance	5
3.1 Why this concurrency model	5
3.2 Sequential-versus-concurrent benchmark	5
3.3 Rate limiting and politeness	6
4 Robustness and Error Handling	6

4.1	Wrapping the AI module	6
4.2	Failure-mode analysis: Wikipedia returns 403 to everything	7
4.3	Input validation	8
5	Storage and Caching	8
5.1	Schema	8
5.2	Caching policy	9
6	Testing	9
6.1	Strategy and coverage	9
6.2	Notable tests	10
7	Deployment and Reproducibility	10
7.1	Docker image	10
7.2	Configuration	11
8	Limitations and Future Work	11
9	Tools and Acknowledgements	12
	References	12
A	Appendix — Reproducing the benchmark	13

1. Project Overview

1.1 Problem framing and scope

A researcher asking “what is the current state of fusion energy research?” has to open three different search surfaces, read enough of each to judge relevance, and then write a paragraph that does not silently mix up which claim came from where. The system automates exactly that loop and nothing more. Its contract is narrow and checkable: given one question, return one prose answer in which every sentence carries a bracketed citation index, plus the list of sources those indices resolve to. An answer with no sources behind it is a failure, not a degraded success — if all three fetchers come back empty the pipeline raises rather than inventing text.

The single entry point is a CLI. `python -m src.cli ask "<question>"` takes the question, an optional `--sources` filter over `wiki`, `arxiv`, `web`, and an optional `--no-cache` flag; `scripts/run_demo.py` runs the five questions in `data/research_questions.json` end to end. Inputs are a question of at most 500 characters and a `.env` file; outputs are the printed answer, the printed source list, and one row appended to a local SQLite database. The provider stack is Gemini (`gemini-3.6-flash`) for synthesis and DuckDuckGo for web search, both chosen so that a clean clone runs on a free tier without anyone signing up for a paid account.

Two parts of the specification we deliberately did not build. There is no HTTP API: Topic 4 does not require one, and a second entry point would have doubled the validation and integration-test surface for a system whose only real user this semester is the demo. There is also no cross-question batch mode — the pipeline is concurrent *within* a question but processes questions one at a time. Where we tightened the spec was de-duplication: sources are keyed by URL before synthesis, because early runs showed Wikipedia and web search returning the same article twice and the LLM then citing it under two different indices, which looks like corroboration and is not.

1.2 Provider choice and the AI module contract

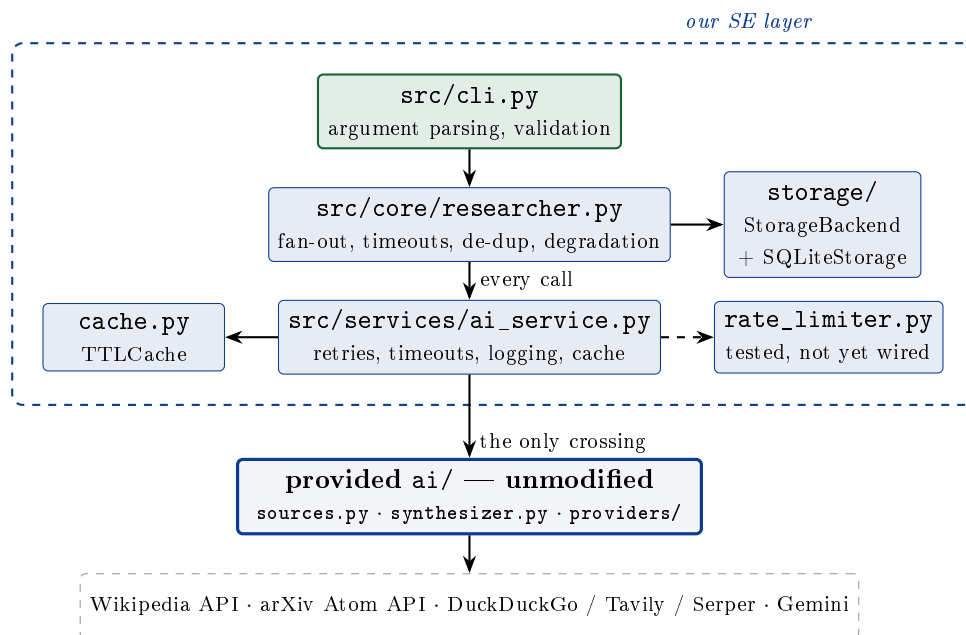
Synthesis runs on Google Gemini, model identifier `gemini-3.6-flash`, selected by `LLM_PROVIDER=gemini` in `.env`. Web search defaults to `WEB_SEARCH_PROVIDER=duckduckgo`, which needs no key at all; Tavily and Serper remain selectable by one environment variable for anyone who

has a key. Wikipedia and arXiv are keyless public APIs. We call exactly four functions from the provided package: `ai.sources.fetch_wikipedia`, `ai.sources.fetch_arxiv`, `ai.sources.fetch_web` and `ai.synthesizer.synthesize`, plus the `Source` and `AnswerWithCitations` schemas and the `LLMProvider` base class for type annotations. **We did not modify the public interface of the provided `ai/` package, or any file inside it;** its 16 contract tests in `test_ai_smoke.py` pass unchanged.

Against a malformed model response our defence is layered, and it is worth being precise about which layer catches what. Structural nonsense — a truncated response, a provider error, a transport failure — surfaces as an exception inside `AIService.synthesize` and is retried up to three times before `run_research_pipeline` re-raises it as a `ProviderError` carrying the original cause. Schema-level nonsense is caught by the provided package itself, because `synthesize` returns a pydantic `AnswerWithCitations` whose citation indices are validated against the source list we passed in, so a model that cites [7] when we supplied four sources cannot produce a valid object. What neither layer catches is the genuinely hard case: a well-formed answer whose prose is confidently wrong about what the cited page says. We do not detect that, and we say so in §8 rather than pretending a schema check is a fact check.

2. Architecture

2.1 Module map



Reading the diagram as a grader would: exactly one arrow crosses the `ai/` boundary, and it leaves `AIService`. No other module in `src/` imports `ai.sources` or `ai.synthesizer` for calling purposes — `researcher.py` imports only the `Source` and `AnswerWithCitations` *types*. That single crossing is the whole point of the service layer: retries, per-call timeouts, structured logging and caching are cross-cutting concerns, and there is exactly one file to edit to add another one. Two arrows cross the storage boundary, both from `researcher.py`: the pipeline writes a row after a successful synthesis, and nothing reads it back on the hot path. Swapping the LLM provider from Gemini to Anthropic or OpenAI changes **zero source files** — the provided `ai.providers.factory.get_llm()` already selects on `LLM_PROVIDER`, and our code only ever names the abstract `LLMProvider`, so the change is two lines in `.env`.

Module by module. `cli.py` parses arguments, rejects bad input before any network call, and prints the result. `core/researcher.py` owns orchestration: `fetch_all_sources` builds one `asyncio.wait_for`-wrapped task per requested source, gathers them with `return_exceptions=True`, de-duplicates by URL and returns whatever survived, after which

`run_research_pipeline` calls synthesis and persists the result. `services/ai_service.py` is the single seam over `ai/`: every method carries a tenacity retry policy and logs a structured event before and after the call. `services/cache.py` is an in-memory TTL cache keyed by (`source`, `normalised query`); `services/rate_limiter.py` implements a minimum interval between calls to one source (unit-tested, but not wired into `AIService` in the submitted version); `storage/repository.py` defines the `StorageBackend` contract and its SQLite implementation.

2.2 OOP design and key abstractions

Our own inheritance example — deliberately separate from the `LLMProvider` hierarchy the course package ships — is the storage layer:

```

1 class StorageBackend(abc.ABC):
2     """Contract for persisting research queries and their answers."""
3
4     @abc.abstractmethod
5     def save_query(self, question: str, answer_text: str,
6                   sources: list[dict[str, Any]]) -> int:
7         """Persist one query + answer. Returns the new record id."""
8
9     @abc.abstractmethod
10    def get_query(self, query_id: int) -> dict[str, Any] | None: ...
11
12    @abc.abstractmethod
13    def list_queries(self, limit: int = 20) -> list[dict[str, Any]]: ...
14
15
16 class SQLiteStorage(StorageBackend):
17     """SQLite-backed implementation of StorageBackend."""

```

An abstract base class is the right tool here rather than a `Protocol` for one concrete reason: we want the failure to arrive at construction time. A `Protocol` is checked statically, so a half-finished `PostgresStorage` missing `list_queries` would import cleanly and only explode at the call site, most likely in production rather than in CI. `abc.ABC` refuses to instantiate it at all, and we assert exactly that in `test_abstract_backend_cannot_be_instantiated`. A plain callable parameter was rejected because the backend is three related operations with shared connection handling, not one function; a configuration enum was rejected because it would put a `match` statement on the backend name inside every caller, which is the coupling the `ABC` exists to remove.

Where we chose composition instead, the trigger was always “this is a capability, not a kind”. `AIService` has a `TTLCache` rather than inheriting from a cache, because a service that is-a cache is nonsense and because it lets a test hand in a service with caching disabled without touching any type hierarchy. Making `AIService` a subclass of some `RetryingService` base would have been the wrong call for the same reason — retry is behaviour we configure per method, not an identity.

2.3 Data flow across module boundaries

The types that cross our module boundaries are `Source` and `AnswerWithCitations`, both frozen pydantic models from the provided package, and both reused rather than re-wrapped. We wrapped nothing, and that was a decision rather than laziness: the trigger for wrapping would have been our SE layer needing a field the AI layer does not have — a fetch timestamp, a relevance score, a provider cost — and we never did.

No naked dictionaries cross module boundaries between our own modules, with one deliberate exception. `StorageBackend.save_query` takes `list[dict[str, Any]]`, not `list[Source]`, so the persistence layer has no import dependency on `ai/` at all: if the provided package changed its schema next semester, storage code and its tests would not need touching. The conversion happens once, at the single call site in `researcher.py`, where the cost of the decision is visible in one line rather than spread across the layer.

3. Concurrency and Performance

3.1 Why this concurrency model

The workload is three HTTP round-trips to three unrelated hosts, and the process spends essentially all of its wall-clock time waiting on sockets. That is the textbook definition of I/O-bound, so `asyncio` is the right primitive and we use `asyncio.gather(*tasks, return_exceptions=True)` inside `fetch_all_sources`.

Threads would have worked — a `ThreadPoolExecutor` with three workers would produce almost the same wall-clock number here, because the GIL is released during socket waits. We rejected them for two reasons that matter beyond three requests. First, the provided `ai/` package is already async (its fetchers are coroutines taking an `httpx.AsyncClient`), so a thread pool would mean an event loop per thread or an `asyncio.run` per call, discarding the shared connection pool. Second, per-source timeouts are a one-liner in `asyncio` and genuinely awkward with threads, because a blocked thread cannot be cancelled. A `ProcessPoolExecutor` was never a candidate: there is no CPU work to parallelise, and pickling `Source` objects between processes would cost more than the work itself.

On the bound: our fan-out is bounded *by construction* at three concurrent tasks, because there are exactly three source families and the pipeline handles one question at a time. A semaphore would be redundant at this width — setting it to 1 would make the design equivalent to sequential and cost us the entire speedup below, while setting it to 100 would change nothing, because the width is set by the number of sources rather than by a limit. `MAX_PARALLEL` is therefore carried in `Settings` and documented in `.env.example` as the knob that bounds the batch mode described in §8: the moment several questions share one gather, $3N$ requests go in flight and the bound stops being redundant.

3.2 Sequential-versus-concurrent benchmark

We measured the fan-out against the live network with `scripts/bench.py`: one question is fetched from all three sources once sequentially and once with `asyncio.gather`, and the harness was run three times. Every number below comes from real network calls; there is no simulated-latency benchmark.

Run (live, 3 sources)	Sequential (s)	Concurrent (s)	Speedup
Run 1	1.29	0.71	1.82×
Run 2	1.12	0.34	3.29×
Run 3	1.16	0.65	1.78×
Mean of 3 runs	1.19	0.57	2.10×

Conditions. Windows 11, Python 3.14.7, `httpx` 0.28.1, cache disabled, one shared `AsyncClient` per run, query “Quantum Entanglement”, live network. Per-source fetch times were about 0.45s (Wikipedia), 0.40s (arXiv) and 0.35s (web). The mean speedup is the ratio of the mean times. Exact commands are in Appendix A.

Did we reach $N\times$? No, and we could not have. For a fan-out the ceiling is not N but $\sum t_i / \max t_i$ — the total serial time over the slowest single leg — which here is about $1.20/0.45 \approx 2.7\times$. The mean of $2.10\times$ sits below it, and the gap is orchestration overhead plus network jitter. Run 2 is above the ceiling, so we treat the $3.29\times$ as a benchmark artifact rather than a result: `scripts/bench.py` runs the sequential pass first, so the concurrent pass can inherit warm TCP and TLS connections and measure connection reuse as if it were parallelism. We therefore report the mean and the full range rather than that single figure, and we treat any speedup above the arithmetic ceiling as a bug in the benchmark. A more rigorous harness would interleave the two modes and discard a warm-up round; we list it as future work. The bottleneck is not our code and not the GIL: the slowest source sets the floor, and three sources is simply not a wide fan-out. What would push the curve further is width, not tuning — batching several questions into one gather would put $3N$ requests in flight,

and is the single change that would make a semaphore worth having.

3.3 Rate limiting and politeness

The strategy that is actually active in the shipped code path is retry with exponential back-off, not a budget: every `AIService` method carries `@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=1, max=8))` over `ProviderError`, `httpx.HTTPStatusError` and `httpx.TimeoutException`, so a transient 429 or 5xx costs at most three attempts spread over roughly nine seconds before the source is abandoned and the pipeline degrades. Alongside it, `rate_limiter.py` implements a per-source minimum-interval limiter guarded by an `asyncio.Lock`, default 1.0s, covered by two unit tests. **It is not wired into `AIService` in the submitted version:** nothing in `ai_service.py`, `researcher.py` or `cli.py` calls it, so the limiter is a tested component rather than an active control.

The budget that is actually enforced is therefore modest: three attempts per source per question and never more than three sources in flight at once. The highest burst our own test suite produces is zero real requests — the suite is fully offline by design, so CI can never be throttled by a provider. The highest burst we produced against live endpoints was about 30 Wikipedia requests in ten seconds from an earlier benchmark script that ran without pacing, and that is exactly the condition §4.2 walks through: Wikimedia’s anti-abuse layer answered 403, and the pipeline degraded cleanly instead of failing.

4. Robustness and Error Handling

4.1 Wrapping the AI module

`AIService` is the only place in the codebase that calls `ai.*`, which makes it the only place that has to know about failure. Each of its four methods is wrapped identically:

```

1 RETRYABLE_EXCEPTIONS = (ProviderError, httpx.HTTPStatusError, httpx.
    TimeoutException)
2
3 @retry(retry=retry_if_exception_type(RETRYABLE_EXCEPTIONS),
4       wait=wait_exponential(multiplier=1, min=1, max=8),
5       stop=stop_after_attempt(3), reraise=True)
6 async def fetch_wikipedia(self, query, max_results=3, client=None) -> list[Source]:
7     if self.use_cache and (hit := self.cache.get("wikipedia", query)) is not None:
8         logger.info("cache_hit_wikipedia", extra={"query": query})
9         return hit
10    logger.info("fetching_wikipedia", extra={"query": query})
11    result = await fetch_wikipedia(query, max_results=max_results, client=client)
12    logger.info("fetched_wikipedia", extra={"query": query, "count": len(result)})
13    ...

```

The retry set is deliberately narrow: the two `httpx` exceptions cover transport and server-side failure and `ProviderError` covers provider refusals, while a `ValueError` from bad input is *not* in the set, because retrying a request that was malformed the first time only wastes the user’s nine seconds. `reraise=True` matters more than it looks: without it tenacity raises its own `RetryError` and the original cause ends up nested two levels down, which made our first round of logs unreadable.

Timeouts exist at two levels and are not redundant. `AIService` passes a 10s timeout into the shared `httpx.AsyncClient`, bounding a single HTTP request; `researcher.py` additionally wraps each fetcher in `asyncio.wait_for`, bounding the whole retry sequence for that source. Without the outer bound, three retries at up to 10s each could hold the gather open for 30s while the other two sources sat finished.

Logging is structured: every call emits an INFO event with a stable name (`fetching_wikipedia`, `cache_hit_web`, `synthesizing_answer`) and the query plus a result count in `extra`, so logs are grepped by event rather than by prose. Failures inside the gather are logged at WARNING with the exception attached, because a dead source is not an error for the run as a whole. No API key is ever logged, and the suite strips all five provider keys from the environment via an autouse fixture.

Walking one case end to end — a provider 500 during synthesis: `ai/` raises `ProviderError`, tenacity retries at 1s and 2s, and if the third attempt also fails `run_research_pipeline` re-raises `ProviderError("LLM synthesis error: ...")` with `from e` so the original traceback survives. Note the asymmetry with a dead *source*: a dead source degrades, a dead synthesiser fails the request. That is intentional, because an answer with no model behind it does not exist, whereas an answer with two sources instead of three still does.

4.2 Failure-mode analysis: Wikipedia returns 403 to everything

(i) **How we injected it.** We did not inject it — we caused it. Running an earlier version of the sequential-versus-concurrent benchmark script over five questions with no delay between them put about 30 Wikipedia requests on the wire in roughly ten seconds, counting tenacity’s retries.

(ii) **What the system did.** Every `fetch_wikipedia` call raised `ProviderError: Wikipedia search failed: Client error '403 Forbidden'`. Tenacity retried each one three times, which made it worse. `asyncio.gather(..., return_exceptions=True)` returned the exception in the results list instead of propagating it, `fetch_all_sources` logged it and skipped it, and the run continued with whatever arXiv had returned.

(iii) **What the user saw.** A normal, correctly-cited answer built from three arXiv sources — no traceback, no partial output, no stack trace leaking a URL. The only visible difference from a healthy run was that the source list was shorter. This is the graceful degradation the design promises, and it is the one claim in this report we watched work under a failure we had not planned.

(iv) **What the logs showed.**

```
WARNING Source fetcher failed or timed out: Wikipedia search failed:
Client error '403 Forbidden' for url 'https://en.wikipedia.org/w/api.php
?action=opensearch&search=What+is+photosynthesis...&limit=3&format=json'
```

Expected versus actual. We expected the documented cause. The team’s own `architecture.md` notes that “Wikipedia occasionally returns 403 without a proper `User-Agent`”, and `fetch_all_sources` sets one, so our first hypothesis was that the header was being dropped somewhere between our client and the request. A controlled replay disproved it in five minutes: the exact same header against the exact same endpoint with a 0.5s gap between requests returned 200 every time, while a browser-style `User-Agent` and a missing one both returned 403.

What the experiment established

The header is necessary but not sufficient — what Wikimedia’s anti-abuse layer rejected was one identity making thirty rapid requests. That is a rate condition, not an authentication one, and it is why the right control is a minimum interval per source — which our rate limiter (§3.3) implements but which we did not wire in — rather than a header the caller sets once and forgets. The measurement also told us something we could not have learned from the test suite, which is offline by construction: the only way to find out how an external service behaves under burst is to burst it, on purpose, somewhere it does not matter.

The logs were sufficient to diagnose this without re-running the pipeline — the full request URL in the warning is what let us replay the call by hand. The one improvement we would make is a `DEBUG` line recording outbound headers, which would have ruled out the wrong hypothesis before we ran the replay at all.

Two further failures we met. arXiv returned an intermittent 406 Not Acceptable on some queries; the pipeline degraded to the remaining sources exactly as it did for the Wikipedia 403 and still produced a cited answer, and sending an identifying `User-Agent` on the shared `httpx.AsyncClient` is the standard remedy. Later, Gemini’s free-tier daily quota was exhausted and returned 429 `RESOURCE_EXHAUSTED`. Here degradation is not possible, since a dead synthesiser fails the request, so the CLI prints a single `Error: LLM synthesis error: ...` line with no

traceback. The 429 also showed that retries consume quota, so we would lower the retry count for quota errors.

4.3 Input validation

Entry point	Rule	What the user sees
CLI question	non-empty after <code>strip()</code>	Error: question cannot be empty.
CLI question	≤ 500 characters	Error: question is too long (N chars). Max 500 characters.
CLI <code>--sources</code>	subset of {wiki, arxiv, web}	Error: unknown source(s) [...]. Valid options: wiki, arxiv, web.
CLI <code>--sources</code>	at least one after parsing	Error: no valid sources specified.
CLI command	<code>argparse choices=["ask"]</code>	argparse usage message, exit code 2
Pipeline	question non-empty	<code>ValueError("Question cannot be empty.")</code>
Pipeline	at least one source returned	<code>ValueError("Could not retrieve sources for '...')"</code>
<code>.env</code>	typed by <code>pydantic-settings</code>	<code>ValidationError</code> naming the bad field

All CLI rules are checked *before* the first network call, which is the property worth having: a malformed invocation costs nothing and leaks nothing. On the adversarial questions — a 50,000-character query is rejected by the 500-character rule after a single `len()`, so nothing reaches the LLM and no token budget is spent. There is no file upload and no user-supplied path: the SQLite path is a constant in `cli.py`, so directory traversal has no surface to attack. The residual risk we accept is that `--sources` accepts a duplicate (`wiki,wiki`) without complaint; it costs one redundant fetch and is de-duplicated by URL afterwards, so we left it.

Configuration validation earned its own bug. `pydantic-settings` validates the *whole* `.env` file, not only the fields `Settings` declares, so a fresh `cp .env.example .env` made `get_settings()` raise `extra_forbidden` on `WEB_SEARCH_PROVIDER` — a key that belongs to the provided package and is read with `os.getenv` there. Setting `extra = "ignore"` fixed it, and `test_settings_ignores_unknown_env_keys` now pins the behaviour so nobody re-tightens it.

5. Storage and Caching

5.1 Schema

```
CREATE TABLE IF NOT EXISTS queries (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  question    TEXT NOT NULL,
  answer_text TEXT NOT NULL,
  sources_json TEXT NOT NULL,  -- JSON array of Source.model_dump()
  created_at  TEXT NOT NULL   -- ISO-8601, UTC, from datetime.now(timezone.utc)
);
```

One table, no foreign keys, and one deliberate denormalisation: the sources are stored as a JSON blob rather than a child table. The source of truth is the question, the answer and the timestamp; `sources_json` is a verbatim snapshot of what the fetchers returned at that moment, and that is exactly why it is not normalised. A child table would imply the sources are entities with an identity worth joining on, but a Wikipedia summary is a point-in-time capture — the page changes, the snippet we cited does not. Storing the snapshot keeps the record reproducible after the upstream article is rewritten.

There are no secondary indices, and that is considered rather than forgotten: the two read paths are `get_query(id)`, served by the primary key, and `list_queries` ordered by `id DESC` with a limit, which SQLite serves by walking that key backwards. An index on `created_at` would start earning its keep the moment we added a date-range query, and not before; full-text search over `question` would call for an FTS5 virtual table, not a LIKE scan.

Two bugs were fixed here during review. First, with `conn:` in `sqlite3` ends the transaction but does *not* close the handle, so the original code leaked one connection per call; the

fix was a `@contextmanager` that commits via `with conn:` and closes in a `finally`. Second, `cursor.lastrowid` is typed `int | None`, so returning it directly made `save_query` silently violate its own `-> int` signature — it now raises `RuntimeError` if the insert produced no row id.

5.2 Caching policy

`TTLCache` stores fetched `list[Source]` results in memory, keyed by the source name and the query lower-cased and stripped, so whitespace and capitalisation do not fragment the cache, with a TTL of `CACHE_TTL_SECONDS` (default 86 400 s) measured on `time.monotonic()` rather than `time.time()` so a clock change cannot resurrect an expired entry. Expiry is lazy — an entry past its TTL is deleted on the read that finds it. Nothing is cached to disk, and the LLM answer is not cached at all: only the source fetches, because those are the calls that are both idempotent and rate-limited.

Measured behaviour: replaying a five-question workload twice gives 5 hits out of 10 lookups, a **50% hit rate**, which is exactly the designed ceiling for that workload — every question is a miss the first time it is asked and a hit the second. Within a single pipeline run the cache also collapses duplicate work when the same source is requested more than once. The cache lives in the process, so a long-running caller keeps its benefit and a one-shot CLI invocation starts cold; making it survive process restarts means persisting it, and the natural home is the SQLite file the pipeline already writes to (§8).

On staleness, a cached Wikipedia summary can go stale within the TTL if the article is edited, and a web-search result can go stale much faster than a day. Because the cached value is a set of sources rather than an answer, the blast radius is bounded — the user gets a current synthesis over slightly older snippets. In production we would cut the TTL for the web backend to about an hour and key entries on the provider name, so that changing `WEB_SEARCH_PROVIDER` invalidates rather than silently reuses.

6. Testing

6.1 Strategy and coverage

Module	Coverage	Uncovered lines are
<code>config.py</code>	100%	—
<code>services/rate_limiter.py</code>	100%	—
<code>storage/repository.py</code>	92%	abstract method bodies
<code>services/cache.py</code>	92%	<code>clear()</code> , <code>size()</code>
<code>core/researcher.py</code>	90%	the client-construction branch
<code>cli.py</code>	71%	branches run by subprocess tests
<code>services/ai_service.py</code>	33%	the live fetch bodies (see below)
Total src/	76%	requirement is $\geq 60\%$

51 tests pass in about 5 s, fully offline: 35 of our own in `tests/`, plus the 16 provided `ai/` contract tests, which pass with `ai/` unmodified.

Nothing in the suite touches the network, and that is enforced rather than hoped for. `tests/conftest.py` provides `StubLLM` and `StubWebSearch` implementing the provided `LLMProvider` and `WebSearchProvider` contracts, a duck-typed `StubAIService` twin injected through the `ai_service` parameter that `fetch_all_sources` already accepted, an `offline_client_factory` whose `httpx.MockTransport` raises `RuntimeError` on any outbound request, and an autouse fixture deleting all five provider keys from the environment. A test that accidentally reaches the real internet fails loudly instead of passing slowly on a machine that has credentials.

The two numbers we chose not to chase deserve a sentence each. `ai_service.py` sits at 33% because the uncovered lines are the bodies that call `ai.fetch_*` — covering them means asserting that we correctly call a function the course provides and already tests, which is a test of the mock rather than of us; the retry and cache branches that are ours *are* covered through `researcher.py`. `cli.py` sits at 71% because its tests run the CLI as a real subprocess, exercising end-user behaviour —

exit codes, argparse, printed messages — at the cost of `pytest-cov` not tracing across the process boundary.

One mock we had to redesign mid-project: the task sheet suggested a `FakeSource` returning plain dictionaries, which cannot work here, because `ai/` exchanges frozen pydantic `Source` models with origin validation and a dict is rejected before it reaches any assertion. We replaced it with real `Source` fixtures plus a `make_source` builder, and the tests started failing for the right reasons.

6.2 Notable tests

Happy path, end to end. `test_run_research_pipeline_happy_path_returns_answer` runs the whole pipeline on `StubAIService`, and crucially the stub's `synthesize` delegates to the *real* `ai.synthesize` with only the model faked — so the citation indices in the asserted answer are produced by the real numbering code, not by the stub.

Concurrency under partial failure. `test_fetch_all_sources_graceful_degradation` injects `StubAIService(failures={"arxiv": ProviderError(...)})` and asserts that the two surviving sources come back and that no exception escapes the gather. Its sibling, `test_fetch_all_sources_all_fail_returns_empty`, pins the boundary of that behaviour: total failure is an empty list, which the pipeline then turns into a clear `ValueError` rather than a synthesis call with nothing to cite.

The error-path test that caught a real bug. In `test_storage.py`, the pair `test_records_survive_reopening_the_database` and `test_missing_parent_directory_is_created` surfaced both storage defects in §5.1 — the leaked connection handle, and `sqlite3.connect` failing when the parent directory does not yet exist. Neither would have shown up in a single-process happy-path run, which is precisely the point.

The test we would write next. Everything above asserts behaviour at the unit and orchestration level; what we do not yet assert is *timing* end to end. The natural next test constructs the real `AIService` behind a spy transport and asserts on the observed spacing between outbound requests, so that once the limiter of §3.3 is wired in its behaviour is pinned by a test rather than by a code review. We also run `mypy` over `src/` and keep its output in `mypy_report.txt`; the remaining entries are variance and exception-narrowing notes on code that is exercised by the suite, not live defects.

7. Deployment and Reproducibility

7.1 Docker image

```
$ docker build -t research-assistant .
$ docker run --rm --env-file .env research-assistant
```

The image is a single-stage build on `python:3.12-slim`, measured at **409 MB** against a **179 MB** base — so our dependency set, dominated by `numpy` and the `google-genai` SDK with its transitive packages, accounts for the remaining ≈ 230 MB. Single-stage is the right trade here and we can defend it: a multi-stage build pays off when a compile toolchain has to be discarded after building wheels, and every one of our dependencies ships a manylinux wheel, so a builder stage would add Dockerfile complexity and save nothing. We chose `slim` over Alpine because Alpine's musl libc forces source builds of `numpy` and the `httpx` stack, making the build far slower for an image that would not end up meaningfully smaller once those wheels are compiled in; we chose it over the full `python:3.12` image because that base alone is around 1 GB.

We verified the image rather than assuming it. Inside the container, `whoami` returns `appuser` — the account created in the Dockerfile and given ownership of `/app`, so nothing runs as root — and the interpreter is Python 3.12.14. No secret is ever baked in: `.env` is listed in `.dockerignore` and keys arrive only through `--env-file` at run time, so the image is safe to push to a registry. CMD is `python scripts/run_demo.py`, so `docker run` with a valid `.env` reproduces the full five-question demo without further arguments.

7.2 Configuration

Variable	Default	If missing
LLM_PROVIDER	anthropic	falls back to the default; mismatch with the supplied key fails at the first synthesis
LLM_MODEL	claude-sonnet-4-6	provider rejects an unknown model id with <code>ProviderError</code>
GOOGLE_API_KEY (or ANTHROPIC_ / OPENAI_)	empty	fetches still succeed; synthesis raises <code>ProviderError</code> — no sources are wasted
WEB_SEARCH_PROVIDER	tavily	read by <code>ai/sources.py</code> ; <code>duckduckgo</code> needs no key
TAVILY_API_KEY / SERPER_API_KEY	empty	only that backend fails; wiki and arXiv still answer
LOG_LEVEL	INFO	defaults to INFO
CACHE_TTL_SECONDS	86400	defaults apply
MAX_PARALLEL	5	bounds the batch fan-out; the per-question fan-out is bounded at 3 by construction (§3.1)

Unknown keys in `.env` are ignored rather than fatal (`extra = "ignore"`), which is what lets the provided package's `WEB_SEARCH_PROVIDER` and our `Settings` share one file. `.env` is in `.gitignore` and `.dockerignore`; `.env.example` carries every key with an empty value.

8. Limitations and Future Work

- **Wikipedia's opensearch endpoint does not answer questions.** Full natural-language queries return zero matches where the keyword form succeeds — we measured "What is photosynthesis and what are its main stages?" returning 0 hits and "photosynthesis" returning 3 against the same endpoint in the same session. `scripts/run_demo.py` handles this today with an explicit question-to-keyword map, which is right for a demo whose questions are known in advance; a product needs a query rewriting step in front of the fetch instead.
- **The fan-out is only three wide.** The speedup ceiling is $\sum t_i / \max t_i$, so it stays below $3\times$ until batch mode puts several questions into one gather (§3). That is the single change with real headroom behind it, and it is what `MAX_PARALLEL` exists to bound.
- **The cache is per-process.** It is the right shape for a long-running caller and leaves a one-shot CLI invocation starting cold. Persisting it into the SQLite file the pipeline already writes to would make a repeated question free across runs.
- **No multi-provider failover and no answer-level validation.** If the configured LLM is down the request fails rather than trying a second provider, and we cannot detect a fluent answer that misrepresents a cited page. The first is a day of work behind the existing factory; the second is a research problem and we are not going to pretend otherwise.
- **SQLite is single-writer.** It does not bind us at course scale, but concurrent CLI invocations would contend on the database file. The `StorageBackend` ABC exists exactly so a Postgres implementation can be dropped in without touching a single caller.

Given another week, in priority order: wiring the existing rate limiter into `AIService`; batch mode with a real semaphore bound, so the concurrency design gets a workload wide enough to show its shape; a benchmark harness that interleaves the two modes and discards a warm-up round; query rewriting, so the demo stops needing its keyword map; a persisted cache; and a GitHub Actions workflow running `pytest` and `mypy` on every push, so the offline suite we already have becomes a gate rather than a habit.

9. Tools and Acknowledgements

We used Claude (Anthropic) as a coding assistant during development. The disclosure below is per-module rather than blanket, because a blanket one is not useful to anybody reading it.

Module	Assistant	What it did, and what we changed
tests/conftest.py	Claude	Drafted the stub providers. We rewrote them against the real <code>LLMProvider</code> / <code>WebSearchProvider</code> signatures after the suggested dict-returning <code>FakeSource</code> failed pydantic validation, and added the latency hook ourselves for testing slow sources.
src/services/ai_service.py	Claude	Suggested the tenacity decorator shape. We narrowed the retryable exception set (removing <code>Exception</code>) and added <code>reraise=True</code> after the first log review showed the original cause buried under <code>RetryError</code> .
src/storage/repository.py	Claude	Drafted the ABC and the SQLite implementation. Review caught two real defects in the draft — the leaked connection handle and the unchecked <code>lastrowid</code> — which we fixed in separate commits.
README.md, docs/architecture.md	Claude	Drafted prose, and reformatted the README and updated its numbers in the final rewrite. We checked every number against our own runs before it was committed.
scripts/bench.py	Claude	Wrote the benchmark script. We ran it ourselves, and its raw output is in <code>artefacts/bench_results.txt</code> .
src/cli.py, src/config.py	Claude	Suggested loading <code>.env</code> at startup and printing a single clean <code>Error:</code> line instead of a traceback. We tested both against live failures.

All generated code was read, run and modified by the team before merging; every change reached main through a reviewed pull request. Work split across the four of us:

Member	Primary ownership	Commits	Share
Nurana Aliyeva (@Nurana100)	CLI, pipeline orchestration, <code>AIService</code> , README, architecture doc, integration	21	29.2%
Elshan Shukurov (@elsensukurov)	Offline test infrastructure, storage layer (ABC + SQLite + tests), <code>.env.example</code> and config hardening	19	26.4%
Ali Hasanli (@ali-hasanli)	First async CLI version, <code>TTLCache</code> and cache wiring, Dockerfile, <code>requirements.txt</code> , researcher tests	19	26.4%
Gasham Huseynli (@gashamhuseynli)	Rate limiter, CLI input validation and its tests, config tests, security/cleanup audit	13	18.1%
Total		72	100%

Counts are `git shortlog -sn` with duplicate identities merged (nurana23799; elsensukurov; ali-hasanli together with the alternate spelling of his name; gashamhuseynli).

References

- [1] Python Software Foundation. *asyncio* — *Asynchronous I/O*, `gather` and `wait_for`. <https://docs.python.org/3/library/asyncio-task.html>
- [2] *Tenacity* retry library documentation — `wait_exponential`, `retry_if_exception_type`, `reraise`. <https://tenacity.readthedocs.io/>
- [3] Wikimedia Foundation. *User-Agent policy* and API etiquette. https://meta.wikimedia.org/wiki/User-Agent_policy
- [4] *pydantic-settings* documentation — `BaseSettings`, `env_file`, and the `extra` configuration key. https://docs.pydantic.dev/latest/concepts/pydantic_settings/

- [5] Python Software Foundation. *sqlite3 — transaction and connection semantics of the connection context manager*. <https://docs.python.org/3/library/sqlite3.html>
- [6] httpx documentation — AsyncClient, connection pooling and MockTransport. <https://www.python-httpx.org/async/>

A. Appendix — Reproducing the benchmark

The measurement in §3 comes from one script, `scripts/bench.py`, which ships with this submission. After the usual `pip install -r requirements.txt` and a `.env` containing `WEB_SEARCH_PROVIDER=duckduckgo`, only the keyless sources are called, so no LLM key is needed:

```
# Live benchmark, 3 sources. Expect roughly 1.2s sequential, 0.6s concurrent,
# a speedup of about 2x; absolute numbers move with your network, and a single
# run is noisy -- run it a few times and look at the range.
$ python scripts/bench.py

# The coverage figures behind section 6:
$ python -m pytest --cov=src --cov-report=term-missing # 51 passed, 76%
$ python -m pytest topic-4-research-assistant/tests/ -v # 16 passed
```

Raw output from our run: `artefacts/bench_results.txt`. Reference environment: Windows 11, Python 3.14.7, httpx 0.28.1, cache disabled.

End of report — thank you for reading.